GanitSūtram

Complete Technical Documentation v1.0

Author: Jawahar R Mallah

Organisation: AITDL | aitdl.com

Date: VS 2082 | Gregorian: 2026-03-08

"Ancient Mathematics. Modern Platform."

Table of Contents

1. Platform Overview
2. Architecture
3. Core Math Engine
4. Backend API
5. Database
6. Authentication & Security
7. Frontend Modules
8. Gamification System
9. Knowledge Graph
10. Search System
11. CMS
12. Deployment
13. Testing
14. Attribution & Legal

- Appendices

SECTION 1 — PLATFORM OVERVIEW

1.1 Vision & Mission

GanitSūtram is a living knowledge ecosystem built on Vedic mathematics, pattern intelligence, and the ancient science of numbers. Mission: make Vedic and classical mathematics accessible, interactive, and intellectually engaging for students, teachers, researchers, and curious minds worldwide.

1.2 Platform Summary

Site	Domain	Purpose
Portal	ganitsutram.com	Homepage, gate, search, CMS
Discoveries	discover.ganitsutram.com	Mathematical discovery browser
Learning	learn.ganitsutram.com	Courses, practice, leaderboard
Knowledge Map	map.ganitsutram.com	Interactive D3 knowledge graph
Research Lab	lab.ganitsutram.com	Experimental Vedic tools
Solver	solve.ganitsutram.com	Live computation engine

1.3 Technology Stack

Layer	Technology
Frontend	Vanilla JS, HTML5, CSS3, D3.js
Backend	Node.js, Express.js
Database	SQLite (better-sqlite3), FTS5
Auth	JWT (15min) + Refresh Tokens (30d)

Email	Nodemailer (SMTP)
Realtime	Server-Sent Events (SSE)
Deploy	Vercel (frontend) + Railway (backend)
i18n	Vanilla JS engine (EN, HI, SA)
PWA	Web App Manifest + Service Workers
Search	SQLite FTS5 + BM25 ranking

1.4 Build History

Phase	Description	Tests
1	Foundational Backend & Schema Setup	0
2	Core Math Engine implementation	24
3	Authentication & Security Layers	18
4	Live Solver API & UI	12
5	Practice Module & Progress Tracking	16
6	Pattern Engine & Discoveries	18
7	Gamification (Points & Badges)	0
8	School & Admin Portal	10
9	Analytics & Dashboards	8
10	Security Audit & Rate Limiting	12
11	Leaderboard & Rankings	12
12	CMS (Content Management System)	12
13	Notifications & SSE Realtime	14

14	Knowledge Graph Backend & Seeds	0
15	Knowledge Graph Visualisation	0
16	Search System (FTS5 + UI)	10
17	Search/Graph Integration & Testing	13
18	PWA, i18n Expansion	0
19	Final UI Polish & Testing	0
20	Launch Preparation (SEO/Sitemaps)	0
Total		**153**

SECTION 2 — ARCHITECTURE

2.1 Folder Structure

```
ganitsutram/
├── backend/
│   ├── api/
│   │   ├── admin.js, analytics.js, cms.js, concepts.js, discoveries.js
│   │   ├── graph.js, health.js, leaderboard.js, notifications.js
│   │   ├── patterns.js, practice.js, search.js, sitemap.js, solve.js
│   │   └── user-progress.js
│   ├── auth/
│   │   ├── auth-routes.js, auth-service.js, jwt-utils.js, password-utils.js
│   ├── database/
│   │   ├── admin-repository.js, analytics-repository.js, badge-repository.js
│   │   ├── cms-repository.js, concepts-repository.js, db.js, discoveries-repo
│   │   ├── graph-repository.js, leaderboard-repository.js, migrations.js
│   │   ├── notification-repository.js, pattern-repository.js, practice-reposi
│   │   ├── progress-repository.js, search-repository.js
│   ├── middleware/
│   │   ├── rate-limiter.js, require-auth.js, require-role.js
│   │   ├── sanitiser.js, security-headers.js
│   ├── services/
│   │   ├── admin-service.js, analytics-service.js, badge-service.js
```

```
│   │   ├── cms-service.js, email-service.js, graph-service.js
│   │   ├── leaderboard-service.js, notification-service.js, search-service.js
│   │   ├── sse-manager.js
│   ├── tests/
│   │   ├── (13 test suite files), test-runner.js
│   └── server.js
├── config/
│   └── .env
├── core/
│   ├── auth-core/
│   │   └── role-definitions.js
│   ├── math-engine/
│   │   ├── digital-root.js, multiply-by-11.js, nikhilam.js
│   │   ├── pattern-engine.js, squares-ending-5.js, urdhva.js
├── docs/
├── scripts/
└── websites/
    ├── discoveries/, knowledge-map/, learning/
    ├── portal/, research-lab/, solver/
    └── ui-core/
        ├── css/, fonts/, locales/, js/
```

2.2 Layer Architecture

```
User Browser
  ↓ HTTPS
Vercel CDN (6 static sites)
  ↓ fetch() API calls
Railway (Express backend — api.ganitsutram.com)
  ↓ imports
Core Math Engine (pure functions)
  ↓ queries
SQLite Database (23 tables)
```

2.3 Architecture Rules

- 5 root folders only (core, backend, websites, docs, config)
- Pure functions only in core/ (no DOM, no HTTP)
- Frontend never imports core/ directly (API only)
- All styles in ui-core/ (never in site folders)

- CommonJS (module.exports) for backend modules

2.4 Design Tokens

Token	Value	Description
`--bg-deep`	`#040110`	Dark Navy Background
`--fg-main`	`#ffffff`	Primary White Text
`--accent-primary`	`#ff5500`	Vibrant Vedic Orange
`--accent-secondary`	`#ffb300`	Golden Accent
`--font-logo`	`'Unbounded'`	Display Font
`--font-main`	`'Space Grotesk'`	Primary Body Font
`--font-ancient`	`'Noto Serif Devanagari'`	Sanskrit / Ancient Text Font

SECTION 3 — CORE MATH ENGINE

3.1 Overview

Location: `core/math-engine/` Rules: pure functions, validate→compute→return, CommonJS exports, no dependencies.

3.2 Module Reference

3.2.1 Digital Root (digital-root.js)

Vedic basis: Beejank / casting out nines Functions: `digitalRoot(n)`, `digitalRootWithSteps(n)` Algorithm: sum digits repeatedly until single digit Example: 98 → 17 → 8 Edge cases: 0 returns 0, 9 returns 9 Throws: invalid input

3.2.2 Squares Ending in 5 (squares-ending-5.js)

Vedic sutra: Ekadhikena Pūrvena Functions: `squaresEndingFive(n)`, `...WithSteps(n)` Algorithm: n=X5 → X×(X+1) concat 25 Example: 75^2 → 7×8=56 → 5625 Throws: input not ending in 5

3.2.3 Multiply by 11 (multiply-by-11.js)

Vedic technique: digit insertion method Functions: `multiplyBy11(n)`, `...WithSteps(n)` Algorithm: insert digit sums between digits Example: 57×11 → 5(5+7)7 → 627

3.2.4 Nikhilam (nikhilam.js)

Vedic sutra: Nikhilam Navatashcaramam Dashatah Functions: `nikhilam(a,b)`, `...WithSteps(a,b)` Algorithm: complement from nearest base Example: 98×97, base 100 → 9506

3.2.5 Ūrdhva-Tiryak (urdhva.js)

Vedic sutra: Ūrdhva-Tiryagbhyām Functions: `urdhva(a,b)`, `...WithSteps(a,b)` Algorithm: cross-multiplication by digit columns Example: 23×45 → 1035

3.2.6 Pattern Engine (pattern-engine.js)

Functions (12 total): detectDigitalRootCycle, detectSquarePattern, detectSumOfDigitsPattern, detectPalindromePattern, detectMultiples11Pattern, detectConsecutiveSquareDiff, analyseSequence, getVedicPattern, listVedicPatterns, kaprekarRoutine, fibonacci, fibonacciDigitalRoots Key result: Kaprekar constant 6174 Key result: Fibonacci DR cycle length 24

SECTION 4 — BACKEND API

4.1 Server Configuration

Location: `backend/server.js` Port: 3000 (env: PORT) Middleware stack (in order): `helmet → CORS → express.json (10kb) → sanitiseBody → localeMiddleware → express.static(websites/) → routes`

4.2 Complete API Reference

AUTH (/api/auth/)

Method	Endpoint	Auth	Purpose
POST	/register	public	Create account
POST	/login	public	Login, get tokens
GET	/me	required	Get current user
POST	/logout	required	Revoke refresh token
POST	/refresh	public	Rotate refresh token
GET	/sessions	required	List active sessions
DELETE	/sessions/:id	required	Revoke session
DELETE	/sessions	required	Revoke all sessions
POST	/forgot-password	public	Send reset email
POST	/reset-password	public	Apply new password

(Other key endpoints documented in codebase OpenAPI standard)

- `/api/solve` — execute math engine operations
- `/api/concepts`, `/api/patterns`, `/api/discoveries` — Read domain content
- `/api/practice` — Submit test evaluations
- `/api/user-progress` — Retrieve ranks
- `/api/admin` — Master administration panel
- `/api/analytics` — Event ingest and retrieval
- `/api/leaderboard` — Ranked global arrays
- `/api/notifications` — SSE channel and preference toggles
- `/api/cms` — Content moderation endpoints
- `/api/search` — FTS search indices
- `/api/graph` — D3 node graph vertices and edges
- `/api/sitemap` — Sitemap XML payload generation

4.3 Request/Response Examples

POST /api/solve

```
{
  "operation": "digital_root",
  "operands": [98]
}
// Response:
{
  "data": { "result": 8, "steps": [...] },
  "attribution": "GanitSūtram | AITDL"
}
```

GET /api/search?q=digital+root

```
{
  "data": {
    "query": "digital root",
    "results": [
      {
        "id": "digital-root",
        "type": "discovery",
        "title": "Digital Root",
        "snippet": "Calculate the <mark>digital</mark> <mark>root</mark> using
        "url": "/discoveries/#digital-root"
      }
    ],
    "count": 1
  },
  "attribution": "GanitSūtram | AITDL"
}
```

4.4 Error Response Format

All errors: `{ "error": "message" }` All success: `{ "attribution": "GanitSūtram | AITDL" }`

SECTION 5 — DATABASE

5.1 Configuration

Engine: SQLite via better-sqlite3 Mode: WAL (Write-Ahead Logging) Foreign keys: ON Path: `/data/ganitsutram.db` (production)

5.2 Schema Reference

Tables (in creation order):

1. users
2. progress
3. practice_attempts
4. reset_tokens
5. refresh_sessions
6. schools
7. enrollments
8. discoveries
9. discovery_patterns
10. analytics_events
11. user_badges
12. user_scores
13. point_events
14. cms_content
15. cms_revisions
16. cms_media
17. notifications
18. notification_prefs
19. search_index
20. search_fts (FTS5 virtual)
21. graph_nodes
22. graph_edges
23. search_history

5.3 Repository Pattern

Repositories act as the single source of truth for all database queries. The services layer imports them to map domain logic cleanly without mixing raw SQL into endpoints.

5.4 Entity Relationship Summary

```
users (id)
  ├─ progress (user_id)
  ├─ practice_attempts (user_id)
  ├─ user_scores (user_id)
  ├─ user_badges (user_id)
  ├─ notifications (user_id)
  └─ refresh_sessions (user_id)

cms_content (id)
  ├─ cms_revisions (content_id)
  └─ cms_media (content_id)

graph_nodes (id)
  └─ graph_edges (source, target)
```

SECTION 6 — AUTHENTICATION & SECURITY

6.1 Authentication Flow

Registration → bcrypt(12) hash → JWT(15min) + RefreshToken(30d, SHA-256 hashed in DB)

Token Refresh Flow (Reuse detection):

```
Client sends refreshToken
  → DB lookup by SHA-256 hash
  → if rotated=1: REUSE DETECTED
      → revoke entire family
      → return 401
  → if valid: issue new JWT + new refreshToken
      → mark old as rotated=1
      → return new tokens
```

6.2 Security Layers

Layer	Implementation
Passwords	bcrypt, 12 rounds
JWT	15-minute expiry
Refresh tokens	SHA-256 hashed, 30-day, rotated
XSS	xss() sanitisation on all inputs
SQL injection	Parameterised queries throughout
Rate limiting	express-rate-limit (per route)
Headers	Helmet.js (CSP, HSTS, noSniff)
Body size	10KB limit
Input cap	10^15 maximum numeric input

6.3 Roles & Permissions

Roles: `student` | `teacher` | `parent` | `adult` | `school` | `admin` Admin role dictates complete oversight, including graph alterations and CMS publishing capabilities.

SECTION 7 — FRONTEND MODULES

7.1 ui-core Overview

Single source of truth for all styles and shared JS. Never duplicated into site folders.

7.2 JavaScript Modules Reference

Module	Global Export	Purpose

ganit-ui.js	window.GanitUI	Core init, nav, auth modal
i18n.js	window.GanitI18n	i18n engine
notifications.js	window.GanitNotif.	Bell, SSE, toasts
search.js	window.GanitSearch	Nav search, results page
leaderboard.js	window.GanitLeader.	Podium, table, badges
cms.js	window.GanitCMS	Admin content editor
seo.js	window.GanitSEO	Dynamic meta updates
knowledge-map.js	(inline)	D3 graph visualisation
solver.js	(inline)	Solver UI + history
discoveries.js	(inline)	Discovery browser
profile.js	(inline)	User profile dashboard
analytics.js	(inline)	Admin analytics charts

7.3 CSS Architecture

Files: `main.css`, `portal.css`, `solver.css`, `discoveries.css`, `learning.css`, `knowledge-map.css`, `research-lab.css`, `leaderboard.css`, `notifications.css`, `search.css`, `cms.css`, `i18n.css`, `profile.css`.

7.4 Localisation

Supported locales: en, hi, sa Locale file: `websites/ui-core/locales/{code}.json`
Fallback chain: requested → en → key string

SECTION 8 — GAMIFICATION SYSTEM

8.1 Points System

Integrated across solver interactions, discovery completions, and learning practice exams.

8.2 Badge Catalogue

20 static badges map to domain achievements (e.g. "Vedic Scholar", "Digital Root Master").

8.3 Leaderboard

Types: Global All-Time | Weekly | Monthly Weekly reset: every Monday 00:00 UTC Display names: alias or masked email

SECTION 9 — KNOWLEDGE GRAPH

9.1 Graph Overview

28 seed nodes, 34+ seed edges. Node types: operation, concept, sutra, domain, persona, pattern.

9.3 Edge Type Reference

Types: `requires` | `related` | `demonstrates` | `extends` | `uses` | `teaches` | `opposite`

9.4 API Endpoints

- GET /api/graph — full graph
- GET /api/graph/node/:id — node + neighbours
- GET /api/graph/path — BFS shortest path
- GET /api/graph/search — node label search

SECTION 10 — SEARCH SYSTEM

10.1 Architecture

SQLite FTS5 virtual table (search_fts) with BM25 ranking and document weight boosting. Fallback: LIKE search if FTS returns 0 results.

10.2 Document Types Indexed

discovery, lesson, sutra, concept, operation, pattern, announcement

10.4 Frontend Search

Nav autocomplete (debounced, 300ms) with a dedicated results page at `/portal/search.html`. Highlighting uses native `<mark>` tags.

SECTION 11 — CMS

11.1 Content Types & Workflow

Content Types: discovery, lesson, sutra, concept, announcement. Workflow: Create (draft) → Edit → Publish → Unpublish Admin Interface at `/portal/cms.html`. Live markdown rendering capabilities.

SECTION 12 — DEPLOYMENT

12.1 Infrastructure

Frontend: Vercel (6 static deployments) Backend: Railway (Node.js + SQLite volume)

12.4 Go-Live Sequence

1. api.ganitsutram.com
2. ganitsutram.com
3. discover.ganitsutram.com
4. learn.ganitsutram.com
5. solve.ganitsutram.com
6. map.ganitsutram.com
7. lab.ganitsutram.com

SECTION 13 — TESTING

13.1 Test Framework

Custom test runner: `backend/tests/test-runner.js` No external libraries. Pure Node.js.

13.2 Test Suites

Suite	Tests	Coverage
core-math	24	All 6 math engine modules
auth	18	Register, login, refresh...
solve	12	All operations + validation
practice	8	Questions, check, stats
patterns	10	All pattern endpoints
discoveries	8	CRUD + category filter
progress	8	Tracking + stats
admin	10	School admin flow
analytics	8	Events + dashboard
security	12	XSS, SQLi, headers, roles
leaderboard	12	Points, badges, ranks
cms	12	Full CMS workflow
notifications	14	SSE, prefs, broadcast
search	10	FTS, suggest, popular
graph	13	Nodes, edges, pathfinding

TOTAL	**179**	(Documenting 153 core + extensions)

SECTION 14 — ATTRIBUTION & LEGAL

14.1 Attribution

Every file in the codebase carries: Project: GanitSūtram Author: Jawahar R Mallah Company: AITDL | aitdl.com VS 2082 | 2026-03-08

14.2 Copyright Notice

© GanitSūtram | AITDL | aitdl.com Vikram Samvat: VS 2082 | Gregorian: 2026-03-08 All Rights Reserved. Unauthorized copying, reproduction, redistribution, or reuse of this software, algorithms, or educational content is prohibited without written permission from AITDL.

APPENDICES

Appendix A — Complete File Index

(See section 2.1 File Structure).

Appendix B — API Quick Reference Card

Refer to backend/api/* router files.

Appendix C — Environment Variables Reference

Key variables: `PORT`, `NODE_ENV`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `DB_PATH`, `SMTP_HOST`, `SMTP_USER`.

Appendix D — Vedic Sutras Reference

Sutra	Sanskrit	English	Used In
Nikhilam Navatash...	निखिलं...	All from 9...	nikhilam.js
Ūrdhva-Tiryagbhyām	ऊर्ध्व...	Vertically...	urdhva.js
Ekadhikena Pūrvena	एकाधिकेन...	By one more...	squares-5.js
Ānurūpyena	आनुरूप्येण	Proportionate...	pattern-engine